A NOTES ON

# DATA STRUCTURES AND ALGORITHMS

*with Python*

---

**Somnath Mukherjee**

Assistant Professor
MCA, RERF

April 24, 2026

# Contents


**1 Introduction to Data Structures** **7**
- 1.1 What is Data Structure . . . 7
- 1.2 Classification of Data Structures . . . 7
- 1.3 Algorithms and Abstract Data Types . . . 8
- 1.4 Basic Terminology . . . 9
- 1.5 Need of Data Structures . . . 9
- 1.6 Advantages of Data Structures . . . 10
- 1.7 Detailed Classification of Data Structures . . . 10
- 1.8 Operations on Data Structures . . . 11

**2 Algorithm Analysis** **13**
- 2.1 What is an Algorithm? . . . 13
- 2.2 Characteristics of an Algorithm . . . 13
- 2.3 Approaches in Algorithms . . . 13
- 2.4 Categories of Algorithms . . . 14
- 2.5 Algorithm Analysis . . . 14
- 2.6 Algorithm Complexity . . . 15
- 2.7 Types of Algorithms . . . 15
- 2.8 Asymptotic Analysis . . . 16
- 2.9 Asymptotic Notations . . . 16

**3 Recursion** **18**

**4 Arrays** **19**
- 4.1 Array . . . 19
  - 4.1.1 Array Declaration . . . 19
  - 4.1.2 Programs . . . 19
  - 4.1.3 Complexity of Array Operations . . . 20
  - 4.1.4 Memory Representation of Array . . . 20
  - 4.1.5 Accessing Elements . . . 20
  - 4.1.6 Address Calculation . . . 21
  - 4.1.7 Passing Array to Function . . . 21
- 4.2 Two-Dimensional Arrays . . . 21
  - 4.2.1 Declaration of 2D Array . . . 21

4.2.2 Structure of 2D Array . . . 21
4.2.3 Accessing Elements of 2D Array . . . 22
4.2.4 Initialization of 2D Array . . . 22
4.2.5 Number of Elements . . . 22
4.2.6 Mapping 2D Array to 1D . . . 22
4.2.7 Row Major Ordering . . . 23
4.2.8 Column Major Ordering . . . 23
4.2.9 Address Calculation . . . 23
4.2.10 Traversal of 2D Array . . . 23

**5 Searching** **24**
5.1 What is Searching? . . . 24
5.1.1 Linear Search . . . 24
5.1.2 Binary Search . . . 25

**6 Sorting** **27**
6.1 Introduction . . . 27
6.2 Bubble Sort . . . 27
6.2.1 Working . . . 27
6.2.2 Algorithm . . . 28
6.2.3 Python Implementation . . . 28
6.2.4 Complexity . . . 28
6.2.5 Properties . . . 28
6.2.6 Applications . . . 29
6.3 Selection Sort . . . 29
6.3.1 Working . . . 29
6.3.2 Algorithm . . . 29
6.3.3 Diagram . . . 30
6.3.4 Python Implementation . . . 30
6.3.5 Complexity . . . 30
6.3.6 Properties . . . 30
6.3.7 Advantages . . . 31
6.3.8 Disadvantages . . . 31
6.4 Insertion Sort . . . 31
6.4.1 Working . . . 31
6.4.2 Algorithm . . . 31
6.4.3 Python Implementation . . . 32
6.4.4 Complexity . . . 32
6.4.5 Properties . . . 32
6.4.6 Advantages . . . 32
6.5 Merge Sort . . . 33
6.5.1 Working . . . 33
6.5.2 Algorithm . . . 33
6.5.3 Python Implementation . . . 34
6.5.4 Complexity . . . 35

6.5.5 Properties . . . 35
6.5.6 Advantages . . . 35
6.5.7 Disadvantages . . . 35
6.6 Quick Sort . . . 35
6.6.1 Working . . . 35
6.6.2 Algorithm . . . 36
6.6.3 Partition Algorithm . . . 36
6.6.4 Diagram . . . 36
6.6.5 Python Implementation . . . 37
6.6.6 Complexity . . . 37
6.6.7 Properties . . . 37
6.6.8 Advantages . . . 37
6.6.9 Disadvantages . . . 38
6.7 Heap Sort . . . 38
6.7.1 Working . . . 38
6.7.2 Algorithm . . . 38
6.7.3 Heapify Procedure . . . 38
6.7.4 Diagram . . . 39
6.7.5 Python Implementation . . . 39
6.7.6 Complexity . . . 40
6.7.7 Properties . . . 40
6.7.8 Advantages . . . 40
6.7.9 Disadvantages . . . 40

**7 Linked Lists** **41**
7.1 Why there is a need of linked list? . . . 41
7.2 What is Linked List? . . . 41
7.3 Representation in Python . . . 42
7.4 Types of Linked List . . . 42
7.4.1 Singly Linked List . . . 42
7.4.2 Doubly Linked List . . . 45
7.4.3 Circular Linked List . . . 47
7.4.4 Doubly Circular Linked List . . . 50
7.4.5 Skip List(Advanced) . . . 51
7.5 Complexity of Linked List . . . 51

**8 Stack** **52**
8.1 What is Stack? . . . 52
8.2 Operations on Stack . . . 52
8.2.1 Push Operation . . . 53
8.2.2 Pop Operation . . . 54
8.3 Applications of Stack . . . 54
8.4 Stack Implementation in Python . . . 55
8.4.1 Push Operation . . . 55
8.4.2 Pop Operation . . . 55

**9 Queue** **56**
9.1 What is a Queue? . . . 56
9.2 Basic Representation . . . 56
9.3 Complexity of Queue . . . 56
9.4 Operations on Queue . . . 56
9.5 Types of Queue . . . 57
9.5.1 Linear Queue . . . 57
9.5.2 Circular Queue . . . 57
9.5.3 Priority Queue . . . 57
9.6 Array Representation of Queue . . . 58
9.7 Queue Implementation in Python . . . 58
9.7.1 Using List . . . 58

**10 Trees** **60**
10.1 What is a Tree? . . . 60
10.2 Definition . . . 60
10.3 Properties of Tree . . . 60
10.4 Basic Tree Structure . . . 61
10.5 Basic Terminologies of Tree . . . 61
10.6 Properties of Tree Data Structure . . . 62
10.7 Implementation of Tree . . . 62
10.8 Applications of Tree . . . 63
10.9 Types of Tree Data Structure . . . 63
10.9.1 General Tree . . . 63
10.9.2 Binary Tree . . . 64
10.9.3 Binary Search Tree (BST) . . . 68
10.9.4 AVL Tree . . . 72
10.9.5 B-Tree . . . 75
10.9.6 B+ Tree . . . 76

**11 Binary Search Trees** **78**

**12 Heaps** **79**

**13 Hashing** **80**
13.1 Introduction to Hashing . . . 80
13.2 Hash Function . . . 80
13.3 Collision . . . 81
13.4 Collision Resolution Techniques . . . 81
13.4.1 Separate Chaining . . . 81
13.4.2 Open Addressing . . . 82
13.5 Rehashing . . . 83
13.6 Extendible Hashing . . . 83
13.7 Time Complexity . . . 83

**14 Graphs** **84**
14.1 Introduction . . . 84
14.1.1 Directed and Undirected Graph . . . 84
14.2 Graph Terminology . . . 85
14.3 Graph Representation . . . 86
14.3.1 Adjacency Matrix Representation(Sequential) . . . 86
14.3.2 Adjacency List (Linked Representation) . . . 87
14.4 Graph Traversal Algorithms . . . 88
14.4.1 Breadth First Search (BFS) . . . 89
14.4.2 Depth First Search (DFS) . . . 90
14.5 Spanning Tree . . . 90
14.5.1 Properties of Spanning Tree . . . 90
14.6 Minimum Spanning Tree (MST) . . . 91
14.6.1 Properties of Minimum Spanning Tree . . . 91
14.6.2 Kruskal's Algorithm . . . 91
14.6.3 Prim's Algorithm . . . 92

# Chapter 1

# Introduction to Data Structures

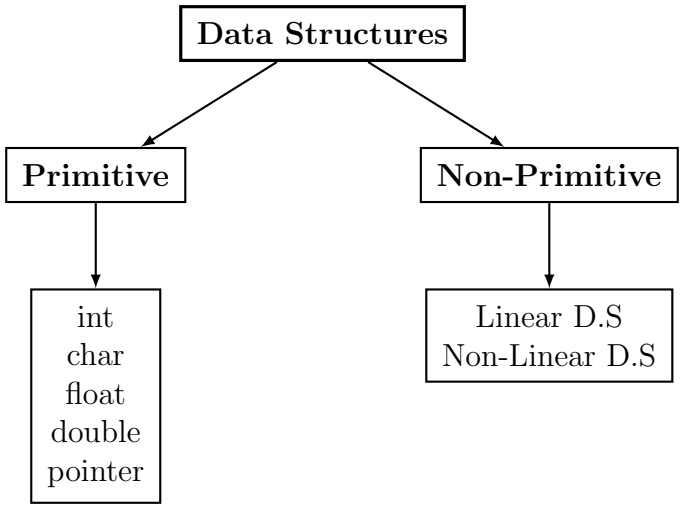
## 1.1 What is Data Structure

A data structure is a way to store and organize data so that it can be used efficiently. As the name suggests, it deals with organizing data in memory.

A data structure is not a programming language such as C, C++, or Java. It is a set of algorithms that can be used in any programming language to structure data in memory.

## 1.2 Classification of Data Structures

Data structures can be broadly classified into:




**Primitive Data Structures:** Primitive data structures are basic data types provided by programming languages.

- int
- char
- float
- double
- pointer

**Non-Primitive Data Structures:** Non-primitive data structures are further divided into:

- Linear Data Structures
- Non-Linear Data Structures

**Linear Data Structures:** Linear data structures store elements in a sequential manner.

The data structures used for this purpose include:

- Arrays
- Linked Lists
- Stacks
- Queues

In linear data structures, each element is connected to only one other element in a linear form.

**Non-Linear Data Structures:** Non-linear data structures are those in which one element can be connected to multiple elements.

**Examples:**

- Trees
- Graphs

In this case, elements are arranged in a hierarchical or non-sequential manner.

## 1.3 Algorithms and Abstract Data Types

Algorithms and Abstract Data Types (ADT) are closely related concepts.

Algorithms
↓
Abstract Data Types
↓
Set of Rules

**Why ADT**

To structure data in memory, a number of algorithms are proposed. These algorithms collectively define Abstract Data Types.

An Abstract Data Type tells **what is to be done**, while a data structure tells **how it is to be done**.

ADT provides the blueprint, while the data structure provides the implementation.

## 1.4 Basic Terminology

**Data:** Data can be defined as elementary values or a collection of values.

**Example:** A student's name and ID represent data about the student.

**Record:** A record is a collection of related data items.

**Example:** Student name, address, course, and marks grouped together form a record.

**File:** A file is a collection of records of the same type.

**Example:** If there are multiple employees, each employee record together forms an employee file.

**Entity and Attribute:**

- An **entity** represents a real-world object.
- An entity contains multiple **attributes**.
- Each attribute represents a specific property of that entity.

## 1.5 Need of Data Structures

As applications are becoming more complex and the amount of data is increasing day by day, several problems arise:

**Processor Speed:** As data grows rapidly, processors may fail to handle billions of files efficiently.

**Data Handling Problem:** Consider an inventory system with 100 items. If a program searches for an item sequentially every time, it slows down the process significantly.

**Multiple Requests:** If thousands of users access data simultaneously, there is a high chance of failure or delay in searching operations.

**Solution:** To solve these problems, data structures are used. Data is organized in such a way that:

- All items are efficiently stored
- Searching becomes faster
- Data access is optimized

## 1.6 Advantages of Data Structures

**Efficiency:** Choosing an appropriate data structure for implementing an ADT improves efficiency in terms of time and space.

**Reusability:** Data structures allow reusability, meaning multiple programs can use the same data structure.

**Abstraction:** Data structures specified by ADT provide a level of abstraction. The user does not need to understand internal implementation details.

## 1.7 Detailed Classification of Data Structures

Data structures can be further classified as follows:

**Primitive Data Structures:**

- int
- char
- float
- double

**Non-Primitive Data Structures:**

**Linear Data Structures:**

- **Static:**
  - Array
- **Dynamic:**
  - Linked List
  - Stack

  - Queue

**Non-Linear Data Structures:**

- Tree
- Graph

## 1.8 Operations on Data Structures

Every data structure contains a set of data elements. Various operations can be performed on these elements to manipulate and process data efficiently.

**1. Traversing:** Traversing means visiting each element of a data structure in order to perform some specific operation such as searching or sorting.

**Example:** To calculate the average marks of a student in multiple subjects, we traverse the entire list of marks, compute the total sum, and divide it by the number of subjects.

**2. Insertion:** Insertion is the process of adding elements to a data structure at any position.

If the size of the data structure is $n$, then we can insert up to $n$ elements depending on capacity.

**3. Deletion:** Deletion is the process of removing an element from a data structure at any location.

If we try to delete an element from an empty data structure, an **underflow** condition occurs.

**4. Searching:** Searching is the process of finding the location of an element within a data structure.

Common searching algorithms include:

- Linear Search
- Binary Search

**5. Sorting:** Sorting is the process of arranging data elements in a specific order.

Some commonly used sorting algorithms are:

- Insertion Sort
- Selection Sort
- Bubble Sort

**6. Merging:** Merging is the process of combining two lists (say list A of size $m$ and list B of size $n$) of similar data types into a single list of size $(m + n)$.

# Chapter 2

# Algorithm Analysis

## 2.1 What is an Algorithm?

An **algorithm** is a process or a set of rules required to perform calculations or solve problems, especially by a computer.

It is not a complete program or code. It is just the logical solution of a problem, which can be represented using a flowchart or pseudocode.

## 2.2 Characteristics of an Algorithm

**Input:** An algorithm has zero or more input values. These inputs are supplied externally.

**Output:** An algorithm produces one or more outputs as the result of processing.

**Unambiguity:** Each step of the algorithm must be clear and precisely defined.

**Finiteness:** An algorithm must terminate after a finite number of steps.

**Effectiveness:** Each operation should be simple, feasible, and executable within a finite time.

## 2.3 Approaches in Algorithms

**Brute Force Algorithm:** Brute force is a general problem-solving technique where all possible solutions are explored to find the required result. It is also known as an exhaustive search approach.

Such algorithms are of two types:

- **Optimizing:** All possible solutions are evaluated, and the best one is selected. The algorithm terminates once the optimal solution is found.

- **Satisficing:** The algorithm stops as soon as a satisfactory or acceptable solution is found.

**Divide and Conquer:** This approach divides a problem into smaller subproblems, solves them independently, and then combines their results to obtain the final solution.

**Greedy Algorithm:** A greedy algorithm makes the best possible choice at each step with the hope of finding the global optimum. It is easy to implement and generally faster, but it does not always guarantee the optimal solution.

## 2.4 Categories of Algorithms

The major categories of algorithms are as follows:

**Sorting:** Algorithms used to arrange elements in a specific order.

**Searching:** Algorithms used to find elements within a data structure.

**Deletion:** Algorithms used to remove existing elements from a data structure.

**Insertion:** Algorithms used to insert new elements into a data structure.

**Update:** Algorithms used to modify existing elements in a data structure.

## 2.5 Algorithm Analysis

An algorithm can be analyzed at two levels:

- Before implementation

- After implementation

There are two types of analysis:

**A Priori Analysis:** A priori analysis is the theoretical analysis of an algorithm, which is done before implementing it.

**A Posteriori Analysis:** A posteriori analysis is the practical analysis of an algorithm. It is performed after implementing the algorithm using a programming language.

## 2.6 Algorithm Complexity

The performance of an algorithm can be measured using different factors.

**Time Complexity:** Time complexity is the amount of time required by an algorithm to complete its execution.

It is usually expressed using **Big-O notation**, which represents the asymptotic behavior of the algorithm.

Time complexity is generally calculated by counting the number of steps required to complete execution.

**Example of Time Complexity**

Consider the following algorithm to calculate the sum of first $n$ numbers:

```
    sum = 0
    for i in range(1, n+1):
    sum = sum + i
    return sum
```

In the above algorithm, the loop runs $n$ times. Therefore, the time complexity of the algorithm is at least $O(n)$.

As the value of $n$ increases, the time complexity also increases.

In general, we consider the **worst-case time complexity**, which represents the maximum time taken for a given input size.

**Space Complexity:** Space complexity is the amount of memory required by an algorithm to solve a problem and produce an output.

Similar to time complexity, space complexity is also expressed using **Big-O notation**.

**Formula:**

$$\text{Space Complexity} = \text{Auxiliary Space} + \text{Input Size}$$

## 2.7 Types of Algorithms

**Search Algorithms:** In daily life, we often search for items. Similarly, in computers, large amounts of data are stored, and when a user requests data, the system searches for it in memory.

There are mainly two techniques to search data:

- Linear Search

- Binary Search

**Sorting Algorithms:** Sorting algorithms are used to arrange elements in a specific order, either ascending or descending.

The comparison operator determines the order of elements.

## 2.8 Asymptotic Analysis

The time required by an algorithm can be analyzed in three cases:

**Worst Case:** It defines the maximum time taken by an algorithm for any input.

**Average Case:** It represents the average time taken by an algorithm for execution.

**Best Case:** It defines the minimum time taken by an algorithm for any input.

## 2.9 Asymptotic Notations

Asymptotic notations are used to represent the running time or time complexity of an algorithm.

**1. Big-O Notation ($O$):** Big-O notation represents the **upper bound** of an algorithm.

It describes the worst-case performance and ensures that the function does not grow faster than the given bound.

**Example:** Let $f(n)$ and $g(n)$ be two functions defined for positive integers. Then,

$$f(n) = O(g(n))$$

if there exist constants $c > 0$ and $n_0$ such that:

$$f(n) \leq c \cdot g(n), \quad \forall n \geq n_0$$

**2. Omega Notation ($\Omega$):** Omega notation represents the **lower bound** of an algorithm.

It describes the best-case performance and gives the minimum time required by an algorithm.

**Example:**

$$f(n) = \Omega(g(n))$$

if there exist constants $c > 0$ and $n_0$ such that:

$$f(n) \geq c \cdot g(n), \quad \forall n \geq n_0$$

**3. Theta Notation ($\Theta$):** Theta notation represents the **tight bound** of an algorithm. It provides both upper and lower bounds and is used when the exact growth rate is known.

**Example:**

$$f(n) = \Theta(g(n))$$

if there exist constants $c_1, c_2 > 0$ and $n_0$ such that:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n), \quad \forall n \geq n_0$$

# Chapter 3

# Recursion

# Chapter 4

# Arrays

## 4.1 Array

An **array** is a collection of elements of similar data types stored at contiguous memory locations.

It is one of the simplest data structures where elements can be accessed directly using their index.

In Python, arrays are generally implemented using **lists**.

### 4.1.1 Array Declaration

In Python, arrays can be created using lists:

```
arr = [10, 20, 30, 40, 50]
```

### 4.1.2 Programs

**Program Without Array:**

```
# Calculating average without array

marks1 = 56
marks2 = 78
marks3 = 89

avg = (marks1 + marks2 + marks3) / 3
print(avg)
```

**Program Using Array:**

```python
# Calculating average using array (list)

marks = [56, 78, 89]

total = 0
for i in range(len(marks)):
total = total + marks[i]

avg = total / len(marks)
print(avg)
```

### 4.1.3 Complexity of Array Operations

**Time Complexity:**

| Operation | Average Case | Worst Case |
|---|---|---|
| Access | $O(1)$ | $O(1)$ |
| Search | $O(n)$ | $O(n)$ |
| Insertion | $O(n)$ | $O(n)$ |
| Deletion | $O(n)$ | $O(n)$ |

**Space Complexity:** The space complexity of an array is $O(n)$.

### 4.1.4 Memory Representation of Array

Elements in an array are stored in contiguous memory locations.

**Indexing Types:**

- Zero-based indexing (Python uses this)
- One-based indexing

Example:

```python
arr = [10, 20, 30, 40, 50]
# Index:  0   1   2   3   4
```

### 4.1.5 Accessing Elements

To access elements of an array:

```python
arr = [10, 20, 30, 40]

print(arr[0])  # 10
print(arr[2])  # 30
```

### 4.1.6 Address Calculation

In low-level languages, the address of an element is calculated as:

$$\text{Address of } A[i] = \text{Base Address} + (i \times \text{Size})$$

Note: Python abstracts memory management, so this calculation is not required explicitly.

### 4.1.7 Passing Array to Function

```
def summation(arr):
total = 0
for i in range(len(arr)):
total += arr[i]
return total

arr = [0, 1, 2, 3, 4]
result = summation(arr)

print(result)
```

**Note:** Python lists are dynamic, meaning their size can grow or shrink during runtime.

## 4.2 Two-Dimensional Arrays

**Definition:**

A two-dimensional (2D) array is an array of arrays. It is used to represent data in the form of rows and columns (matrix form).

In Python, 2D arrays are implemented using nested lists.

### 4.2.1 Declaration of 2D Array

In Python, a 2D array can be declared as:

```
arr = [
[1, 2, 3],
[4, 5, 6],
[7, 8, 9]
]
```

### 4.2.2 Structure of 2D Array

A 2D array of size $m \times n$ can be visualized as:

```
arr = [
[a00, a01, a02],
[a10, a11, a12],
[a20, a21, a22]
]
```

Here:

- $i$ represents row index
- $j$ represents column index

### 4.2.3 Accessing Elements of 2D Array

Elements are accessed using two indices:

```
arr = [
[10, 20, 30],
[40, 50, 60]
]

print(arr[0][1])  # Output: 20
print(arr[1][2])  # Output: 60
```

Here, the first index represents the row and the second represents the column.

### 4.2.4 Initialization of 2D Array

```
rows = 2
cols = 3

arr = [[0 for j in range(cols)] for i in range(rows)]

print(arr)
```

### 4.2.5 Number of Elements

The total number of elements in a 2D array is:

$$\text{Number of Elements} = \text{rows} \times \text{columns}$$

### 4.2.6 Mapping 2D Array to 1D

A 2D array can be stored in memory as a 1D array.

For example, a $3 \times 3$ array:

```
[
[0,1,2],
[3,4,5],
[6,7,8]
]
```

### 4.2.7 Row Major Ordering

In row major order, elements are stored row by row.

```
[0, 1, 2, 3, 4, 5, 6, 7, 8]
```

### 4.2.8 Column Major Ordering

In column major order, elements are stored column by column.

```
[0, 3, 6, 1, 4, 7, 2, 5, 8]
```

### 4.2.9 Address Calculation

For a 2D array $A[m][n]$:

**Row Major Order:**

$$\text{Address}(A[i][j]) = \text{Base Address} + ((i \times n) + j) \times \text{size}$$

**Column Major Order:**

$$\text{Address}(A[i][j]) = \text{Base Address} + ((j \times m) + i) \times \text{size}$$

**Note:** Python handles memory management internally.

### 4.2.10 Traversal of 2D Array

```
arr = [
[1, 2, 3],
[4, 5, 6]
]

for i in range(len(arr)):
for j in range(len(arr[0])):
print(arr[i][j], end=" ")
```

# Chapter 5

# Searching

## 5.1 What is Searching?

Searching is the process of finding a particular element in a list. If the element is present, the search is called successful; otherwise, it is unsuccessful.

There are two commonly used methods:

- Linear Search
- Binary Search

### 5.1.1 Linear Search

Linear search is the simplest searching algorithm, also known as sequential search.

In this method, we traverse the entire list and compare each element with the target value.

It is mainly used when the list is **unsorted**.

#### Algorithm

1. Initialize position $\text{pos} = -1$
2. Traverse the list from index 0 to $n - 1$
3. If element matches the target value:
   - Set $\text{pos} = i$
   - Return position
4. If no match is found, return $-1$

### Python Implementation

```
arr = [5, 10, 23, 40, 12, 0, 14, 13, 30, 97]

item = int(input("Enter item to search: "))

pos = -1

for i in range(len(arr)):
if arr[i] == item:
pos = i
break

if pos != -1:
print("Item found at location", pos)
else:
print("Item not found")
```

### Complexity

| | Best Case | Average Case | Worst Case |
|---|---|---|---|
| Time | $O(1)$ | $O(n)$ | $O(n)$ |
| Space | $O(1)$ | | |

## 5.1.2 Binary Search

Binary search is an efficient searching technique that works on **sorted lists**.

It follows the **divide and conquer** approach, where the list is repeatedly divided into two halves and the middle element is compared with the target value.

**Note:** Binary search works only on sorted data.

### Algorithm

1. Initialize:
   - beg $= 0$
   - end $= n - 1$
2. Repeat while beg $\leq$ end:
   - Compute mid $= \lfloor(\text{beg} + \text{end})/2\rfloor$
   - If $A[\text{mid}] = \text{val}$:
     - Return mid

- Else if $A[\text{mid}] > \text{val}$:
  - Set $\text{end} = \text{mid} - 1$
- Else:
  - Set $\text{beg} = \text{mid} + 1$

3. If not found, return $-1$

### Python Implementation

```
arr = [5, 10, 12, 13, 14, 23, 30, 40, 97]  # sorted list

item = int(input("Enter item to search: "))

beg = 0
end = len(arr) - 1
pos = -1

while beg <= end:
mid = (beg + end) // 2

if arr[mid] == item:
pos = mid
break
elif arr[mid] > item:
end = mid - 1
else:
beg = mid + 1

if pos != -1:
print("Item found at location", pos)
else:
print("Item not found")
```

### Complexity

| | Best Case | Average Case | Worst Case |
|---|---|---|---|
| Time | $O(1)$ | $O(\log n)$ | $O(\log n)$ |
| Space | $O(1)$ | | |

# Chapter 6

# Sorting

## 6.1 Introduction

Sorting algorithms are techniques used to rearrange elements in a list or array in a specific order (ascending or descending).

They are fundamental for efficient data processing and searching.

**Common Sorting Algorithms:**

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Heap Sort

## 6.2 Bubble Sort

Bubble sort is the simplest sorting algorithm that works by repeatedly swapping adjacent elements if they are in the wrong order.

It is called **Bubble Sort** because larger elements move to the end of the array like bubbles rising to the surface.

### 6.2.1 Working

- Compare adjacent elements

- Swap if they are in wrong order
- Repeat for all elements
- After each pass, the largest element moves to the end

### 6.2.2 Algorithm

1. For $i = 0$ to $n - 1$
2. For $j = 0$ to $n - i - 2$
   - If $A[j] > A[j + 1]$, swap them

$$5 \quad 3 \quad 8 \quad 4$$

After Pass: Largest element moves right

Figure 6.1: Bubble Sort Process

### 6.2.3 Python Implementation

```python
arr = [10, 35, 32, 13, 26]

n = len(arr)

for i in range(n):
for j in range(0, n - i - 1):
if arr[j] > arr[j + 1]:
arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted array:", arr)
```

### 6.2.4 Complexity

| Case | Time Complexity | Space Complexity |
|---|---|---|
| Best Case | $O(n)$ | $O(1)$ |
| Average Case | $O(n^2)$ | $O(1)$ |
| Worst Case | $O(n^2)$ | $O(1)$ |

### 6.2.5 Properties

- **Stable:** Yes
- **In-place:** Yes

- **Adaptive:** Yes (optimized version)
- **Not suitable for large datasets**

### 6.2.6 Applications

- When dataset is small
- When simplicity is preferred over efficiency

## 6.3 Selection Sort

Selection Sort is a simple sorting algorithm that repeatedly selects the minimum element from the unsorted portion and places it at the beginning.

### 6.3.1 Working

- Divide the array into sorted and unsorted parts
- Initially, the sorted part is empty
- Find the minimum element from the unsorted part
- Swap it with the first unsorted element
- Move the boundary of sorted part by one

### 6.3.2 Algorithm

1. For $i = 0$ to $n - 2$
2. Set min_index $= i$
3. For $j = i + 1$ to $n - 1$
   - If $A[j] < A[\text{min_index}]$
   - Update min_index $= j$
4. Swap $A[i]$ and $A[\text{min_index}]$

### 6.3.3 Diagram

64 25 12 22

Select minimum (12)

12 25 64 22

After first pass

Figure 6.2: Selection Sort Process

### 6.3.4 Python Implementation

```
arr = [64, 25, 12, 22, 11]

n = len(arr)

for i in range(n):
min_index = i

for j in range(i + 1, n):
if arr[j] < arr[min_index]:
min_index = j

arr[i], arr[min_index] = arr[min_index], arr[i]

print("Sorted array:", arr)
```

### 6.3.5 Complexity

| Case | Time Complexity | Space Complexity |
|---|---|---|
| Best Case | $O(n^2)$ | $O(1)$ |
| Average Case | $O(n^2)$ | $O(1)$ |
| Worst Case | $O(n^2)$ | $O(1)$ |

### 6.3.6 Properties

- **Stable:** No
- **In-place:** Yes
- **Adaptive:** No
- Performs minimum number of swaps

### 6.3.7 Advantages

- Simple implementation
- Requires less number of swaps

### 6.3.8 Disadvantages

- Poor performance on large datasets
- Time complexity is always $O(n^2)$

## 6.4 Insertion Sort

Insertion sort is a sorting algorithm that works similar to sorting playing cards in hand.

It builds the sorted array one element at a time by inserting each element into its correct position.

### 6.4.1 Working

- Assume first element is already sorted
- Pick next element (key)
- Compare it with previous elements
- Shift larger elements to the right
- Insert key at correct position

### 6.4.2 Algorithm

1. For $i = 1$ to $n - 1$
2. Set $key = A[i]$
3. Set $j = i - 1$
4. While $j \geq 0$ and $A[j] > key$
   - $A[j + 1] = A[j]$
   - $j = j - 1$
5. Insert $key$ at $A[j + 1]$

12 31 25 8

Insert 25 between 12 and 31

Figure 6.3: Insertion Sort Process

### 6.4.3 Python Implementation

```
arr = [12, 31, 25, 8, 32, 17]

for i in range(1, len(arr)):
  key = arr[i]
  j = i - 1

while j >= 0 and arr[j] > key:
  arr[j + 1] = arr[j]
  j -= 1

arr[j + 1] = key

print("Sorted array:", arr)
```

### 6.4.4 Complexity

| Case | Time Complexity | Space Complexity |
|---|---|---|
| Best Case | $O(n)$ | $O(1)$ |
| Average Case | $O(n^2)$ | $O(1)$ |
| Worst Case | $O(n^2)$ | $O(1)$ |

### 6.4.5 Properties

- **Stable:** Yes
- **In-place:** Yes
- **Adaptive:** Yes
- Efficient for small or nearly sorted data

### 6.4.6 Advantages

- Simple implementation
- Efficient for small datasets
- Works well for nearly sorted data

## 6.5 Merge Sort

Merge Sort is a sorting algorithm based on the **Divide and Conquer** approach.

It divides the array into smaller subarrays, sorts them recursively, and then merges the sorted subarrays to obtain the final sorted array.

### 6.5.1 Working

- Divide the array into two halves
- Recursively sort both halves
- Merge the two sorted halves

### 6.5.2 Algorithm

**MergeSort(A, beg, end)**

1. If $beg < end$
    - $mid = \lfloor (beg + end)/2 \rfloor$
    - MergeSort(A, beg, mid)
    - MergeSort(A, mid+1, end)
    - Merge(A, beg, mid, end)

**Merge(A, beg, mid, end)**

1. Create two temporary arrays:
    - Left = A[beg ... mid]
    - Right = A[mid+1 ... end]
2. Compare elements of both arrays
3. Copy smaller element into original array
4. Copy remaining elements

38 27 43 3

Divide

38 27 43 3

Merge

3 27 38 43

Figure 6.4: Merge Sort Process

### 6.5.3 Python Implementation

```
def merge_sort(arr):
if len(arr) > 1:
mid = len(arr) // 2

left = arr[:mid]
right = arr[mid:]

merge_sort(left)
merge_sort(right)

i = j = k = 0

while i < len(left) and j < len(right):
if left[i] <= right[j]:
arr[k] = left[i]
i += 1
else:
arr[k] = right[j]
j += 1
k += 1

while i < len(left):
arr[k] = left[i]
i += 1
k += 1

while j < len(right):
arr[k] = right[j]
j += 1
k += 1

arr = [38, 27, 43, 3, 9, 82]
merge_sort(arr)
```

```
    print("Sorted array:", arr)
```

### 6.5.4 Complexity

| Case | Time Complexity | Space Complexity |
|---|---|---|
| Best Case | $O(n \log n)$ | $O(n)$ |
| Average Case | $O(n \log n)$ | $O(n)$ |
| Worst Case | $O(n \log n)$ | $O(n)$ |

### 6.5.5 Properties

- **Stable:** Yes
- **In-place:** No
- **Divide and Conquer:** Yes
- Works well for large datasets

### 6.5.6 Advantages

- Consistent performance
- Suitable for large data
- Good for linked lists

### 6.5.7 Disadvantages

- Requires extra memory
- Not in-place

## 6.6 Quick Sort

Quick Sort is a sorting algorithm based on the **Divide and Conquer** approach. It works by selecting a **pivot** element and partitioning the array such that elements smaller than the pivot are placed on the left and greater elements on the right.

### 6.6.1 Working

- Select a pivot element
- Partition the array into two parts:
  - Elements less than pivot

  - Elements greater than pivot

- Recursively apply Quick Sort on both parts

### 6.6.2 Algorithm

**QuickSort(A, low, high)**

1. If $low < high$
   - $p = \text{Partition}(A, low, high)$
   - QuickSort(A, low, p-1)
   - QuickSort(A, p+1, high)

### 6.6.3 Partition Algorithm

**Partition(A, low, high)**

1. Set pivot $= A[high]$
2. Set $i = low - 1$
3. For $j = low$ to $high - 1$
   - If $A[j] < pivot$
     - $i = i + 1$
     - Swap $A[i]$ and $A[j]$
4. Swap $A[i + 1]$ and $A[high]$
5. Return $i + 1$

### 6.6.4 Diagram

10 7 8 9 1 5

Pivot = 5

1 5 8 9 10 7

After Partition

Figure 6.5: Quick Sort Partitioning

### 6.6.5 Python Implementation

```python
def partition(arr, low, high):
pivot = arr[high]
i = low - 1

for j in range(low, high):
if arr[j] < pivot:
i += 1
arr[i], arr[j] = arr[j], arr[i]

arr[i + 1], arr[high] = arr[high], arr[i + 1]
return i + 1


def quick_sort(arr, low, high):
if low < high:
pi = partition(arr, low, high)

quick_sort(arr, low, pi - 1)
quick_sort(arr, pi + 1, high)


arr = [10, 7, 8, 9, 1, 5]
quick_sort(arr, 0, len(arr) - 1)
print("Sorted array:", arr)
```

### 6.6.6 Complexity

| Case | Time Complexity | Space Complexity |
|---|---|---|
| Best Case | $O(n \log n)$ | $O(\log n)$ |
| Average Case | $O(n \log n)$ | $O(\log n)$ |
| Worst Case | $O(n^2)$ | $O(n)$ |

### 6.6.7 Properties

- **Stable:** No
- **In-place:** Yes
- **Adaptive:** No
- Very fast in practice

### 6.6.8 Advantages

- Faster than most $O(n \log n)$ algorithms in practice

- No extra memory required

### 6.6.9 Disadvantages

- Worst case is $O(n^2)$
- Performance depends on pivot selection

## 6.7 Heap Sort

Heap Sort is a comparison-based sorting algorithm that uses a **Binary Heap** data structure. It first builds a **max heap** and then repeatedly extracts the maximum element to sort the array.

### 6.7.1 Working

- Build a max heap from the input array
- Swap the root (maximum element) with the last element
- Reduce heap size by one
- Heapify the root to maintain heap property
- Repeat until the array is sorted

### 6.7.2 Algorithm

**HeapSort(A)**

1. Build a max heap from array A
2. For $i = n - 1$ down to 1
    - Swap $A[0]$ and $A[i]$
    - Reduce heap size
    - Heapify(A, 0)

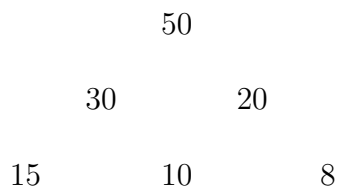
### 6.7.3 Heapify Procedure

**Heapify(A, n, i)**

1. Set largest = i
2. left = $2i + 1$, right = $2i + 2$

3. If left child exists and $A[left] > A[largest]$, update largest
4. If right child exists and $A[right] > A[largest]$, update largest
5. If largest $\neq i$
   - Swap $A[i]$ and $A[largest]$
   - Recursively call Heapify on largest

### 6.7.4 Diagram




Max Heap Structure

Figure 6.6: Binary Heap Representation

### 6.7.5 Python Implementation

```python
def heapify(arr, n, i):
largest = i
left = 2 * i + 1
right = 2 * i + 2

if left < n and arr[left] > arr[largest]:
largest = left

if right < n and arr[right] > arr[largest]:
largest = right

if largest != i:
arr[i], arr[largest] = arr[largest], arr[i]
heapify(arr, n, largest)


def heap_sort(arr):
n = len(arr)

# Build max heap
for i in range(n // 2 - 1, -1, -1):
heapify(arr, n, i)

```

```python
    # Extract elements
    for i in range(n - 1, 0, -1):
    arr[0], arr[i] = arr[i], arr[0]
    heapify(arr, i, 0)


    arr = [12, 11, 13, 5, 6, 7]
    heap_sort(arr)
    print("Sorted array:", arr)
```

### 6.7.6 Complexity

| Case | Time Complexity | Space Complexity |
|---|---|---|
| Best Case | $O(n \log n)$ | $O(1)$ |
| Average Case | $O(n \log n)$ | $O(1)$ |
| Worst Case | $O(n \log n)$ | $O(1)$ |

### 6.7.7 Properties

- **Stable:** No
- **In-place:** Yes
- **Adaptive:** No
- Uses binary heap structure

### 6.7.8 Advantages

- Guaranteed $O(n \log n)$ performance
- No extra memory required

### 6.7.9 Disadvantages

- Not stable
- Slower in practice compared to Quick Sort

# Chapter 7

# Linked Lists

## 7.1 Why there is a need of linked list?

If we declare an array of size 3, we know that all the values of an array are stored in a continuous manner, so all three values of an array are stored sequentially.

Then total memory space occupied by array will be:

$$3 \times 4 = 12 \text{ bytes}$$

**Drawbacks of using array:**

- We cannot insert more than 3 elements in above example because only 3 spaces are allocated.
- In case of array, wastage of memory can occur.
- In array, we are providing fixed size at compile time due to which wastage of memory occurs.

The solution to this problem is to use **linked list**.

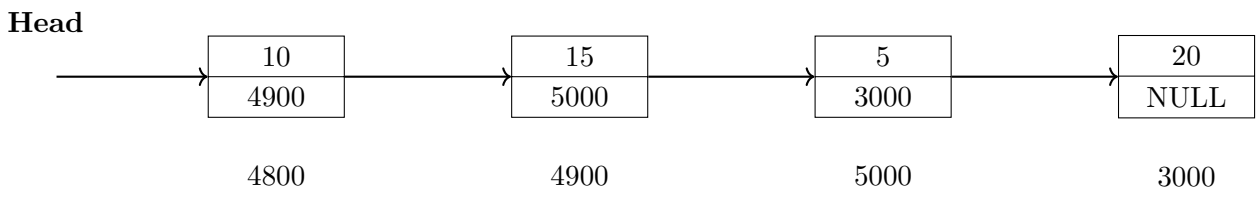
## 7.2 What is Linked List?

A linked list is also a collection of elements, but the elements are not stored in a consecutive location.

A linked list is a collection of nodes in which one node is connected to another node.

Each node consists of two parts:

- Data part
- Address part (reference)




## 7.3 Representation in Python

In Python, we implement linked list using classes.

```
class Node:
def __init__(self, data):
self.data = data
self.next = None
```

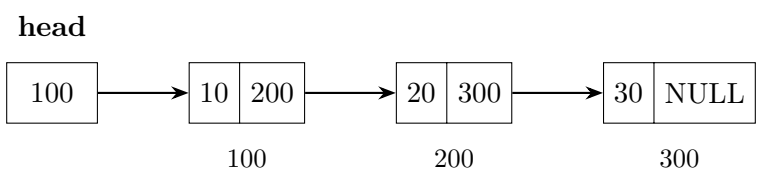
## 7.4 Types of Linked List

### 7.4.1 Singly Linked List

The singly linked list is most common.

It consists of data part and address part (next pointer).

The pointer that holds the address of the first node is known as the head.




**Example:**

```
class Node:
  def __init__(self, data):
    self.data = data
    self.next = None

# Creating nodes
head = Node(10)
second = Node(20)
third = Node(30)

# Linking nodes
head.next = second
```

```
second.next = third
```

### Operations on Singly Linked List

#### 1. Node Creation

- In Python, a node can be created using a class:

```
class Node:
def __init__(self, data):
self.data = data
self.next = None
```

#### 2. Insertion

- **Insertion at Beginning**
  It involves inserting a new element at the front of the list. The new node becomes the new head of the list.

```
def insert_at_beginning(head, data):
new_node = Node(data)
new_node.next = head
return new_node
```

- **Insertion at End**
  The new node is inserted as the last node in the list.

```
def insert_at_end(head, data):
new_node = Node(data)
if head is None:
return new_node
temp = head
while temp.next:
temp = temp.next
temp.next = new_node
return head
```

- **Insertion after a Specific Node**
  We traverse to the desired node and insert the new node after it.

```
def insert_after(head, target, data):
temp = head
while temp:
if temp.data == target:
new_node = Node(data)
new_node.next = temp.next
temp.next = new_node
break
temp = temp.next
```

```
    return head
```

### 3. Deletion

- **Deletion at Beginning**
  It removes the first node of the list.

```
def delete_at_beginning(head):
if head is None:
return None
return head.next
```

- **Deletion at End**
  The last node of the list is removed.

```
def delete_at_end(head):
if head is None or head.next is None:
return None
temp = head
while temp.next.next:
temp = temp.next
temp.next = None
return head
```

### 4. Traversal

- In traversal, we visit each node of the list once.

```
def traverse(head):
temp = head
while temp:
print(temp.data, end=" -> ")
temp = temp.next
print("NULL")
```

### 5. Searching

- In searching, we compare each element with the target value.

```
def search(head, key):
temp = head
position = 0
while temp:
if temp.data == key:
return position
temp = temp.next
position += 1
return -1
```

### 7.4.2 Doubly Linked List

The doubly linked list contains two pointers:

- Next pointer
- Previous pointer

**Representation:**

```
class Node:
def __init__(self, data):
self.data = data
self.next = None
self.prev = None
```

**Operations on Doubly Linked List**

**1. Node Creation**

- A doubly linked list node contains:
  - Pointer to previous node
  - Data
  - Pointer to next node

```
class Node:
def __init__(self, data):
    self.prev = None
self.data = data
self.next = None

# Head node
head = None
```

**2. Insertion**

- **Insertion at Beginning**
  Adding the node at the beginning of the list.

```
def insert_at_beginning(head, data):
new_node = Node(data)
if head is not None:
head.prev = new_node
new_node.next = head
return new_node
```

- **Insertion at End**
  Adding the node at the end of the list.

```
def insert_at_end(head, data):
new_node = Node(data)
if head is None:
return new_node

temp = head
while temp.next:
temp = temp.next

temp.next = new_node
new_node.prev = temp
return head
```

### 3. Deletion and Traversal

- **Deletion at Beginning**
  Removing the node from the beginning of the list.

```
def delete_at_beginning(head):
if head is None:
return None

head = head.next
if head:
head.prev = None

return head
```

- **Deletion at End**
  Removing the node from the end of the list.

```
def delete_at_end(head):
if head is None:
return None

if head.next is None:
return None

temp = head
while temp.next:
temp = temp.next

temp.prev.next = None
return head
```

- **Traversal**
  Visiting each node of the list at least once to perform operations like searching, sorting, or display.

```
def traverse(head):
temp = head
while temp:
print(temp.data, end=" <-> ")
temp = temp.next
print("NULL")
```

**4. Searching**

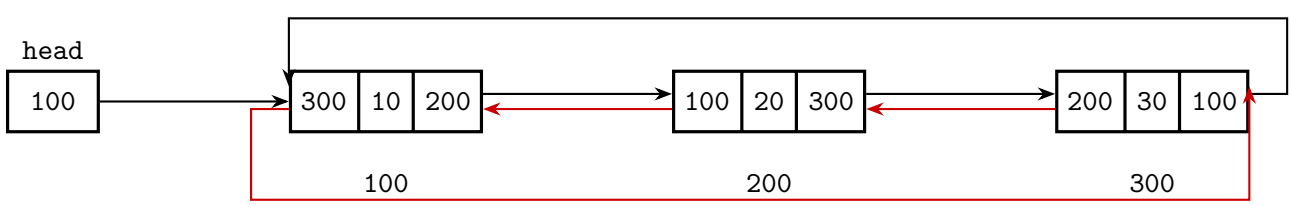
- Comparing each node's data with the target value.
- Returns the position if found, otherwise returns -1.

```
def search(head, key):
temp = head
position = 0

while temp:
if temp.data == key:
return position
temp = temp.next
position += 1

return -1
```

### 7.4.3 Circular Linked List

A circular linked list is a variation of singly linked list.

The last node points back to the first node instead of NULL.




**Example:**

```
head = Node(10)
second = Node(20)
third = Node(30)

head.next = second
```

```
second.next = third
third.next = head   # Circular link
```

### Operations on Circular Linked List

### 1. Node Creation

- Each node contains data and a pointer to the next node.

```
# Head node
head = None
```

### 2. Insertion

- **Insertion at Beginning**
  Adding a node at the beginning while maintaining circular structure.

```
def insert_at_beginning(head, data):
new_node = Node(data)

if head is None:
new_node.next = new_node
return new_node

temp = head
while temp.next != head:
temp = temp.next

new_node.next = head
temp.next = new_node

return new_node
```

- **Insertion at End**
  Adding a node at the end of the list.

```
def insert_at_end(head, data):
new_node = Node(data)

if head is None:
new_node.next = new_node
return new_node

temp = head
while temp.next != head:
temp = temp.next

temp.next = new_node
new_node.next = head
```

```

    return head
```

### 3. Deletion

- **Deletion at Beginning**
  Removing the first node and updating links.

```
def delete_at_beginning(head):
if head is None:
return None

if head.next == head:
return None

temp = head
while temp.next != head:
temp = temp.next

temp.next = head.next
return head.next
```

- **Deletion at End**
  Removing the last node of the list.

```
def delete_at_end(head):
if head is None:
return None

if head.next == head:
return None

temp = head
prev = None

while temp.next != head:
prev = temp
temp = temp.next

prev.next = head
return head
```

### 4. Traversal

- Each node is visited once, stopping when we reach the head again.

```
def traverse(head):
if head is None:
```

```
    return

    temp = head
    while True:
    print(temp.data, end=" -> ")
    temp = temp.next
    if temp == head:
    break
    print("(back to head)")
```

**5. Searching**

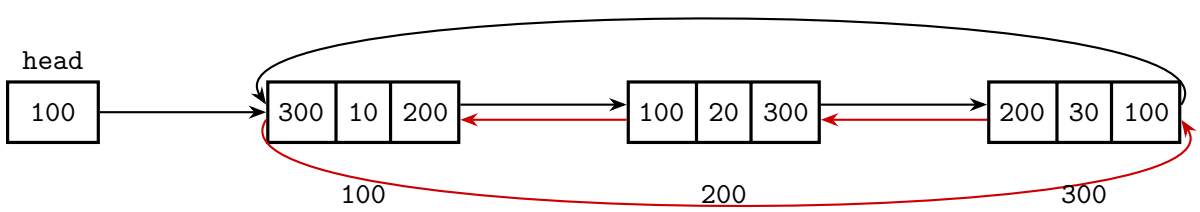
- Compare each node with the target value.
- Returns position if found, otherwise returns -1.

```
    def search(head, key):
    if head is None:
    return -1

    temp = head
    position = 0

    while True:
    if temp.data == key:
    return position
    temp = temp.next
    position += 1
    if temp == head:
    break

    return -1
```

### 7.4.4 Doubly Circular Linked List

The doubly circular linked list has features of both circular linked list and doubly linked list.

The last node connects to the first node and there is no NULL in previous pointer.




**Representation:**

```python
class Node:
def __init__(self, data):
self.data = data
self.next = None
self.prev = None

# Example linking
head = Node(10)
second = Node(20)

head.next = second
second.prev = head

second.next = head
head.prev = second
```

### 7.4.5 Skip List(Advanced)

**What is a Skip List?**

- A skip list is a probabilistic data structure.
- It is used to store elements in a linked list form.
- In a single step, it can skip several elements of the list.
- Due to this ability to skip elements, it is called a *skip list*.

**Structure of Skip List**

- A skip list is built in multiple layers.
- The lowest layer is a simple sorted linked list.
- The upper layers act like *express lanes*, allowing faster traversal.
- These upper layers skip several elements, reducing search time.

## 7.5 Complexity of Linked List

| Singly Linked List | Access | Search | Insertion | Deletion | Space (Worst) |
|---|---|---|---|---|---|
| **Average Case** | $O(n)$ | $O(n)$ | $O(1)$ | $O(1)$ | $O(n)$ |
| **Worst Case** | $O(n)$ | $O(n)$ | $O(1)$ | $O(1)$ | $O(n)$ |

# Chapter 8

# Stack

## 8.1 What is Stack?

A **stack** is a linear data structure that follows the **LIFO (Last-In-First-Out)** principle.

A stack has only one end, whereas a queue has two ends (front and rear).

**Definition**

A stack is a container in which insertion and deletion operations are performed from only one end, known as the **top** of the stack.

**Characteristics**

- Follows LIFO (Last-In-First-Out) principle
- Insertion operation is called **push**
- Deletion operation is called **pop**
- Only one end (top) is accessible

**Properties**

A stack is an **Abstract Data Type (ADT)** with a pre-defined capacity, which means that it can store elements of limited size.

## 8.2 Operations on Stack

The following are the basic operations that can be performed on a stack:

1. **Push()**
   When an element is inserted into a stack, the operation is called **push**. If the stack is full, then an **overflow** condition occurs.

2. **Pop()**
   When an element is removed from the stack, the operation is called **pop**. If the stack is empty, then an **underflow** condition occurs.

3. **Peek()**
   It returns the topmost element of the stack without removing it.

4. **Count()**
   It returns the total number of elements present in the stack.

5. **Change()**
   It modifies the element at a given position in the stack.

6. **Display()**
   It displays all the elements present in the stack.

### 8.2.1 Push Operation

**Definition:** The **push()** operation is used to insert an element into the stack. The insertion is always done at the top of the stack.

**Steps:**

- Before inserting an element, check whether the stack is full.
- If the stack is full, an **overflow** condition occurs.
- Initially, the value of **top = -1**, which indicates that the stack is empty.
- When an element is inserted, increment the top pointer:

$$\text{top} = \text{top} + 1$$

- Insert the element at the new top position.

**Illustration of Push Operation:**

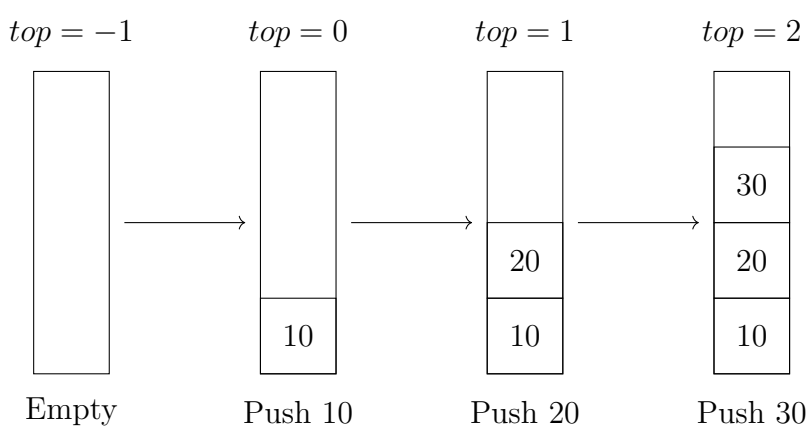


*Fig: Push Operation in Stack*

### 8.2.2 Pop Operation

**Definition:** The **pop()** operation is used to remove an element from the stack. The deletion is always performed from the top of the stack.

**Steps:**

- Before deleting an element, check whether the stack is empty.
- If the stack is empty, an **underflow** condition occurs.
- Access the element pointed to by the **top**.
- After deletion, decrement the top pointer:

$$\text{top} = \text{top} - 1$$

**Illustration of Pop Operation:**

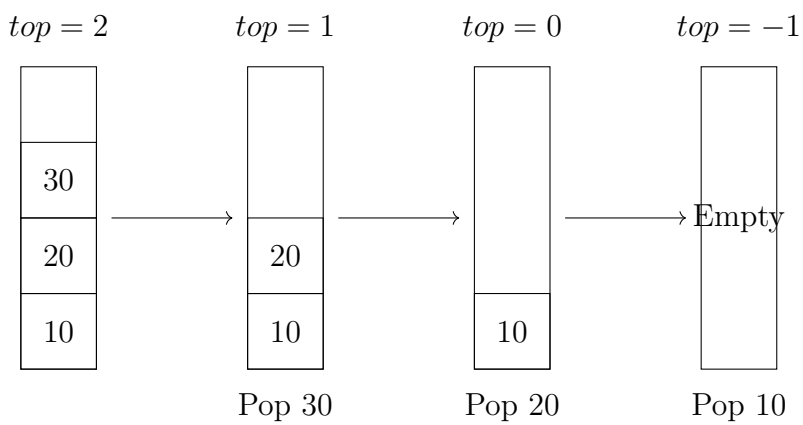



*Fig: Pop Operation in Stack*

## 8.3 Applications of Stack

1. **Recursion**
   Recursion is a process in which a function calls itself. To maintain previous states, the compiler uses a **system stack**, where all function calls are stored.

2. **Depth First Search (DFS)**
   DFS is a graph traversal technique that uses a stack (either explicitly or via recursion).

3. **Backtracking**
   Backtracking is used to solve problems like maze solving. If a wrong path is chosen, we backtrack to the previous state using a stack.

4. **Memory Management**
   Stack is used for managing memory in contiguous blocks, especially for function calls and local variables.

## 8.4 Stack Implementation in Python

### 8.4.1 Push Operation

```
class Stack:
def __init__(self, size):
self.stack = []
self.size = size
self.top = -1

def push(self, item):
if self.top == self.size - 1:
print("Stack Overflow")
return

self.top += 1
self.stack.append(item)
```

Listing 8.1: Push Operation

**Time Complexity:** $O(1)$

### 8.4.2 Pop Operation

```
def pop(self):
if self.top == -1:
print("Stack Underflow")
return None

item = self.stack.pop()
self.top -= 1
return item
```

Listing 8.2: Pop Operation

**Time Complexity:** $O(1)$

# Chapter 9

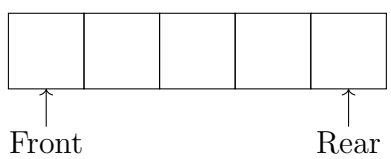
# Queue

## 9.1 What is a Queue?

A **queue** is a linear data structure in which insertion is performed at one end called the **rear**, and deletion is performed at another end called the **front**.

It follows the **FIFO (First-In-First-Out)** principle.

## 9.2 Basic Representation




## 9.3 Complexity of Queue

| Operation | Access | Search | Insertion | Deletion |
|---|---|---|---|---|
| Average | $O(n)$ | $O(n)$ | $O(1)$ | $O(1)$ |
| Worst | $O(n)$ | $O(n)$ | $O(1)$ | $O(1)$ |

## 9.4 Operations on Queue

1. **Enqueue()**
   Inserts an element at the rear of the queue.

2. **Dequeue()**
   Removes an element from the front of the queue.

3. **Peek()**
   Returns the front element without removing it.

4. **Overflow**
   Occurs when the queue is full.

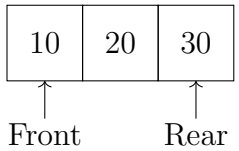
5. **Underflow**
   Occurs when the queue is empty.

## 9.5 Types of Queue

### 9.5.1 Linear Queue

In a linear queue, insertion takes place at the rear and deletion occurs from the front. It strictly follows FIFO.

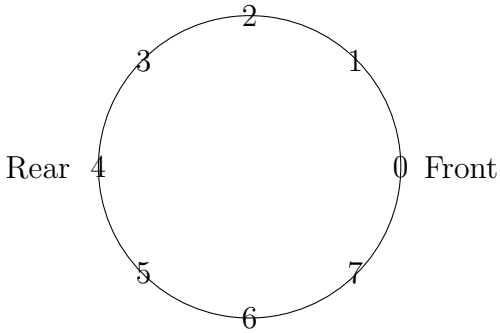
**Drawback:** Memory wastage occurs when front moves forward.




### 9.5.2 Circular Queue

In a circular queue, the last position is connected back to the first position, forming a circular structure.

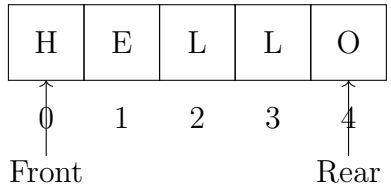
This overcomes the drawback of linear queue by reusing empty spaces.




### 9.5.3 Priority Queue

In a priority queue, each element has a priority. Elements with higher priority are served first. If priorities are equal, FIFO is followed.

## 9.6 Array Representation of Queue




## 9.7 Queue Implementation in Python

### 9.7.1 Using List

```
class Queue:
def __init__(self, size):
self.queue = []
self.size = size
self.front = -1
self.rear = -1

def enqueue(self, item):
if self.rear == self.size - 1:
print("Queue Overflow")
return

if self.front == -1:
self.front = 0

self.rear += 1
self.queue.append(item)

def dequeue(self):
if self.front == -1 or self.front > self.rear:
print("Queue Underflow")
return None

item = self.queue[self.front]
self.front += 1
return item

def peek(self):
if self.front == -1:
return None
return self.queue[self.front]
```

Listing 9.1: Queue Implementation

**Time Complexity:** $O(1)$ for enqueue and dequeue

# Chapter 10

# Trees

## 10.1 What is a Tree?

A **tree** is a non-linear data structure. Unlike arrays, linked lists, stacks, and queues (where elements are arranged sequentially), a tree represents data in a hierarchical manner.

## 10.2 Definition

A tree is a data structure defined as a collection of objects or entities called **nodes**, which are linked together to represent a hierarchical structure.

A tree is a non-linear data structure because it does not store data in a sequential manner. In a tree, elements are arranged in multiple levels.

## 10.3 Properties of Tree

- In a tree data structure, the topmost node is called the **root node**.
- Each node contains some data, and the data can be of any type.
- Each node contains some data and links (or references) to other nodes, which are called **children**.

## 10.4 Basic Tree Structure

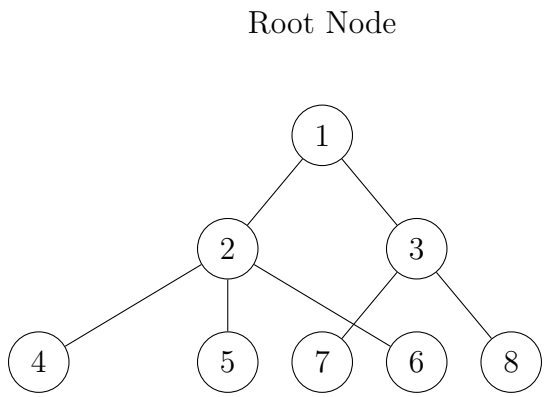



Figure 10.1: Basic Tree Structure

## 10.5 Basic Terminologies of Tree

1. **Link**
   Each connection between two nodes is called a **link**. It represents the relationship between nodes.

2. **Root Node**
   The root node is the topmost node in the hierarchy. It does not have any parent node.

3. **Child Node**
   If a node is a descendant of another node, then it is called a **child node**.

4. **Parent Node**
   If a node contains any sub-node, then that node is called the **parent** of that sub-node.

5. **Sibling**
   Nodes that have the same parent are called **siblings**.

6. **Leaf Node**
   A node that does not have any child is called a **leaf node**. It is the bottom-most node of a tree.

7. **Ancestor Node**
   Any predecessor node on the path from the root to a given node is called an **ancestor node**.

8. **Descendant Node**
   The nodes that are successors of a given node are called its **descendants**.

## 10.6 Properties of Tree Data Structure

1. **Recursive Data Structure**
   A tree is also known as a **recursive data structure**. Recursion means defining something in a self-similar manner. Each subtree of a tree is itself a tree.

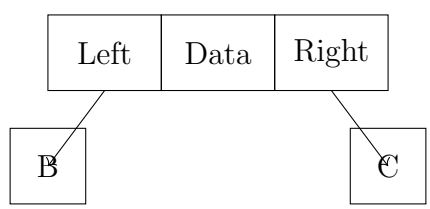
2. **Number of Edges**
   If there are $n$ nodes in a tree, then there are $(n-1)$ edges. Each node, except the root node, has exactly one incoming edge.

3. **Depth of a Node**
   The depth of a node $x$ is defined as the length of the path from the root node to node $x$.
   Each edge contributes one unit length. Therefore, depth can be defined as the number of edges between the root node and node $x$.
   The root node has depth 0.

4. **Height of a Node**
   The height of a node $x$ is defined as the length of the longest path from node $x$ to a leaf node.

## 10.7 Implementation of Tree

The tree data structure can be created dynamically using pointers. The memory representation of a tree node is shown below:




**Structure of Node (C representation):**

```python
class Node:
def __init__(self, data):
self.data = data
self.left = None   # reference to left child
self.right = None  # reference to right child
```

Listing 10.1: Binary Tree Node in Python

The above structure is applicable for **binary trees**, as each node can have at most two children (left and right).

## 10.8 Applications of Tree

1. **Storing Hierarchical Data**
   Trees are used to represent hierarchical data such as file systems, where files and folders are organized in a hierarchical manner.

2. **Organizing Data**
   Trees help in organizing data for efficient insertion, deletion, and searching operations.

3. **Trie**
   A trie is a special type of tree used to store dictionaries. It is efficient for dynamic spell checking and prefix-based searching.

4. **Heap**
   A heap is a tree-based data structure implemented using arrays. It is used to implement priority queues.

## 10.9 Types of Tree Data Structure

### 10.9.1 General Tree

In a general tree, a node can have either 0 or a maximum of $n$ children. There are no restrictions imposed on the degree of a node (number of children a node can contain).

The topmost node in a general tree is known as the **root node**. The children of a parent node form **subtrees**.

There can be $n$ number of subtrees in a general tree. Subtrees in a general tree are unordered, i.e., nodes in a subtree cannot be ordered.

Every non-empty tree has a downward edge, and these edges connect nodes. The nodes that have the same parent are called **siblings**.

**Representation of General Tree:**

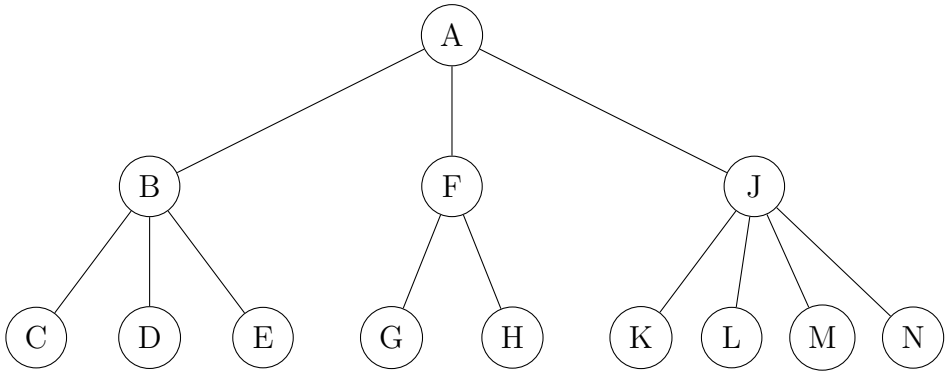



Figure 10.2: General Tree Structure

### 10.9.2 Binary Tree

A **binary tree** is a tree in which each node can have at most two children, called the **left child** and the **right child**.

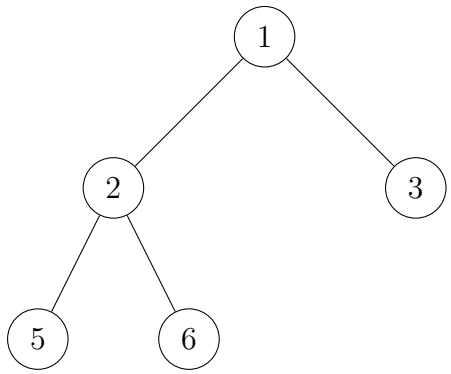



Figure 10.3: Binary Tree

#### Logical Representation of Binary Tree

In a binary tree, each node contains two pointers:

- Left pointer (points to left child)
- Right pointer (points to right child)

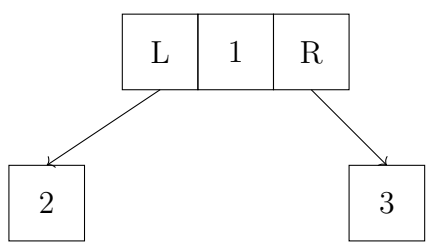



Figure 10.4: Logical Representation of Binary Tree

The nodes $3, 5$, and $6$ are **leaf nodes**, so their left and right pointers are `NULL`.

#### Properties of Binary Tree

- At level $i$, the maximum number of nodes is:

$$2^i$$

- The maximum number of nodes in a binary tree of height $h$ is:

$$2^{h+1} - 1$$

- The minimum number of nodes in a binary tree of height $h$ is:

$$h + 1$$

- If the number of nodes is minimum, then the height of the tree is maximum.
- Minimum height of a binary tree with $n$ nodes:

$$h = \lfloor \log_2 n \rfloor$$

- Maximum height of a binary tree with $n$ nodes:

$$h = n - 1$$

**Types of Binary Tree**

**1. Full (Proper/Strict) Binary Tree**

A **full binary tree** is a binary tree in which each node contains either 0 or 2 children. No node has exactly one child.

**Example:**

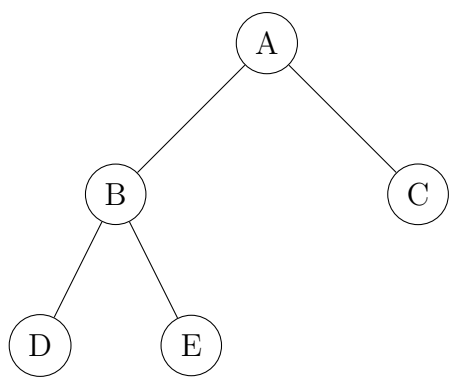



Figure 10.5: Full Binary Tree

**Properties:**

- Maximum number of nodes at height $h$:

$$2^{h+1} - 1$$

- Minimum number of nodes at height $h$ (for full binary tree):

$$2h + 1$$

- Number of leaf nodes:

$$\frac{n+1}{2}$$

- Number of internal nodes:
$$\frac{n-1}{2}$$

**2. Complete Binary Tree**

A **complete binary tree** is a binary tree in which all levels are completely filled except possibly the last level, and the last level is filled from left to right.

**Example:**

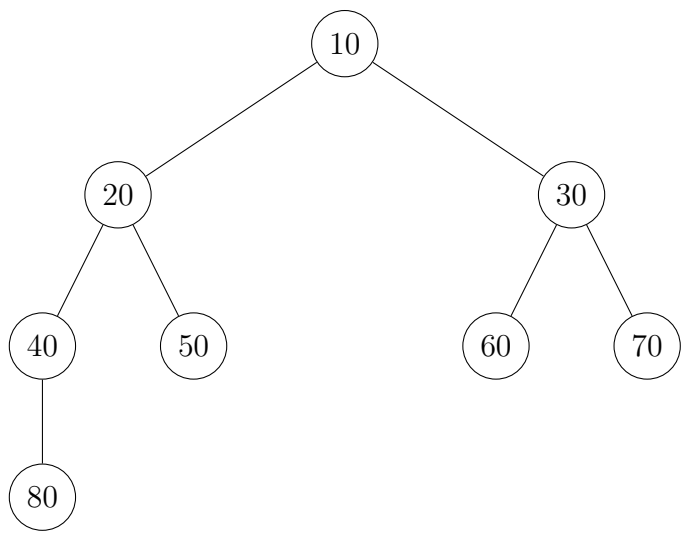



Figure 10.6: Complete Binary Tree

**Properties:**

- Maximum number of nodes at height $h$:
$$2^{h+1} - 1$$
- Minimum number of nodes at height $h$:
$$2^h$$
- Minimum height for $n$ nodes:
$$h = \lfloor \log_2 n \rfloor$$
- Maximum height for $n$ nodes:
$$h = n - 1$$

**3. Perfect Binary Tree**

A **perfect binary tree** is a binary tree in which:

- All internal nodes have exactly two children
- All leaf nodes are at the same level

**Example:**

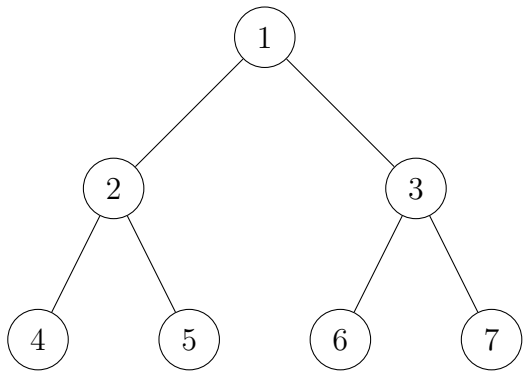



Figure 10.7: Perfect Binary Tree

**Properties:**

- Total number of nodes:
$$n = 2^{h+1} - 1$$
- Number of leaf nodes:
$$2^h$$
- Height of tree:
$$h = \log_2(n+1) - 1$$

**4. Balanced Binary Tree**

A **balanced binary tree** is a tree in which the difference between the heights of the left and right subtrees of any node is at most 1.

**Example:**

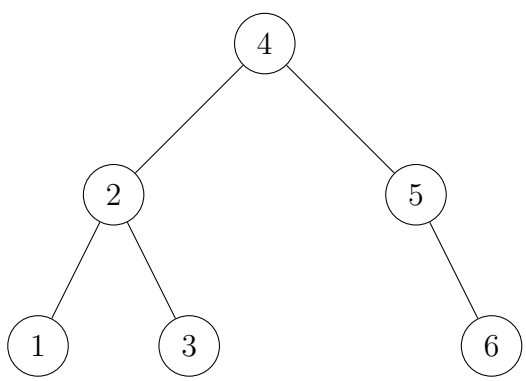



Figure 10.8: Balanced Binary Tree

**Tree Traversal**

Tree traversal is the process of visiting all the nodes in a tree.

There are three types of traversal:

1. **Inorder Traversal (Left → Root → Right)**
2. **Preorder Traversal (Root → Left → Right)**
3. **Postorder Traversal (Left → Right → Root)**

### 10.9.3 Binary Search Tree (BST)

A **Binary Search Tree (BST)** is a binary tree in which nodes are arranged in a specific order.

**Definition:**

- All elements in the left subtree are **less than** the root node.
- All elements in the right subtree are **greater than or equal to** the root node.

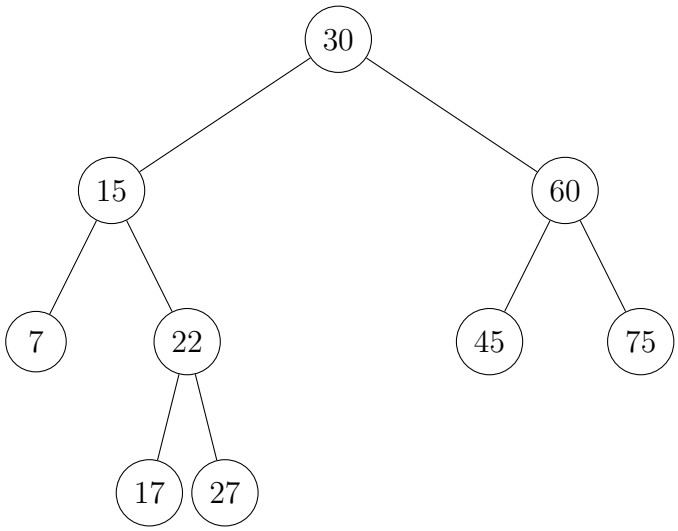



Figure 10.9: Binary Search Tree

**Insertion in BST**

1. Insert the first element as the root node.
2. For each new element:
   - If the element is less than the current node, move to the left subtree.
   - If the element is greater than or equal to the current node, move to the right subtree.

3. Repeat until a NULL position is found.

4. Insert the element at that position.

```
class Node:
def __init__(self, data):
self.data = data
self.left = None
self.right = None

def insert(root, key):
if root is None:
return Node(key)

if key < root.data:
root.left = insert(root.left, key)
else:
root.right = insert(root.right, key)

return root
```

Listing 10.2: BST Insertion

**Example: Construction of Binary Search Tree**

Given elements:
$43, 10, 79, 90, 12, 54, 11, 9, 50$

**Steps:**

1. Insert 43 as root node.

2. Insert 10 (less than 43 $\rightarrow$ left).

3. Insert 79 (greater than 43 $\rightarrow$ right).

4. Insert 90 (greater than 79 $\rightarrow$ right).

5. Insert 12 (greater than 10 $\rightarrow$ right).

6. Insert 54 (less than 79 $\rightarrow$ left).

7. Insert 11 (less than 12 $\rightarrow$ left).

8. Insert 9 (less than 10 $\rightarrow$ left).

9. Insert 50 (less than 54 $\rightarrow$ left).

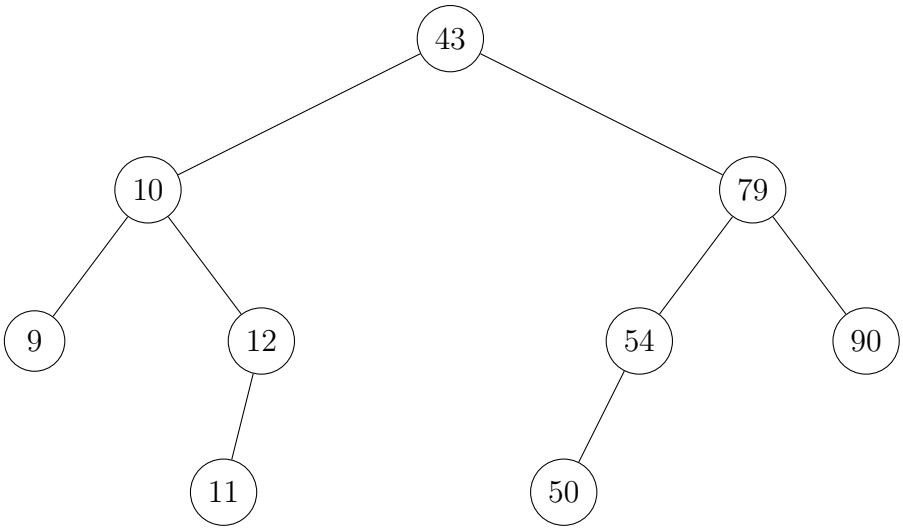
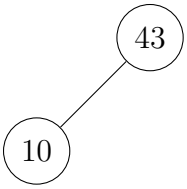



Figure 10.10: Binary Search Tree after all insertions

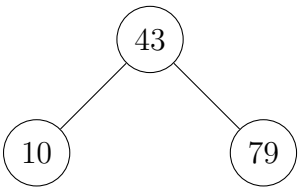
**Step 1:**

**Step 2:**

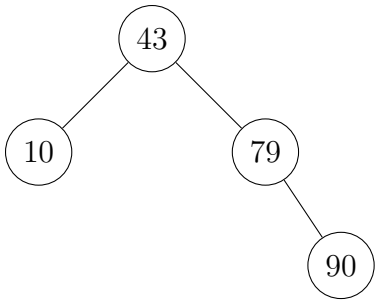



**Step-wise Construction of BST (Continued)**

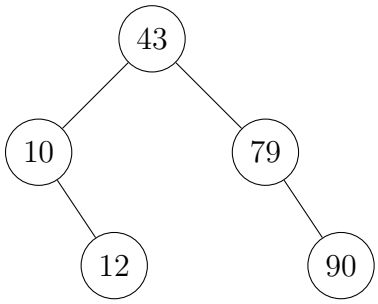
**Step 3: Insert 79**



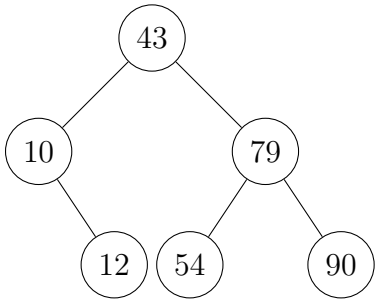

**Step 4: Insert 90**



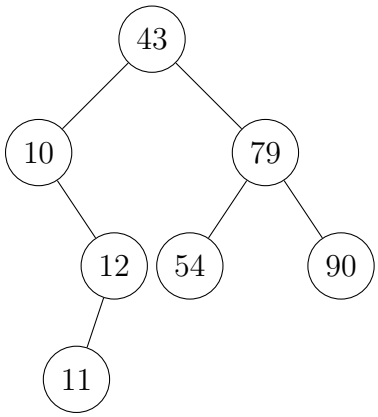

**Step 5: Insert 12**



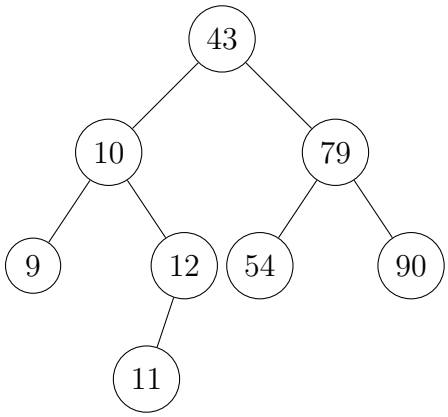

**Step 6: Insert 54**



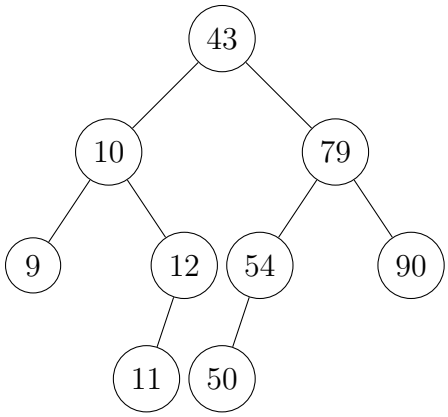

**Step 7: Insert 11**



**Step 8: Insert 9**




**Step 9: Insert 50**




**Operations on Binary Search Tree (BST)**

| S.No. | Operation | Description |
|---|---|---|
| 1. | Searching in BST | Finding the location of a specific element in a Binary Search Tree. |
| 2. | Insertion in BST | Adding a new element to the Binary Search Tree at an appropriate position such that the BST properties are not violated. |
| 3. | Deletion in BST | Removing a specific node from a BST. There are multiple cases in deletion depending on the number of children the node has. |

### 10.9.4 AVL Tree

An **AVL tree** is a self-balancing Binary Search Tree (BST) invented by **G. M. Adelson-Velsky** and **E. M. Landis** in 1962. The name AVL is derived from the initials of its inventors.

An AVL tree is defined as a height-balanced binary search tree in which each node is associated with a **balance factor**.

**Balance Factor:**

$$BF(x) = \text{height(left subtree)} - \text{height(right subtree)}$$

- If $BF = +1$, the left subtree is one level higher than the right subtree.
- If $BF = 0$, both left and right subtrees have equal height.
- If $BF = -1$, the right subtree is one level higher than the left subtree.

For an AVL tree, the balance factor of every node must be in the range:

$$-1 \leq BF \leq 1$$

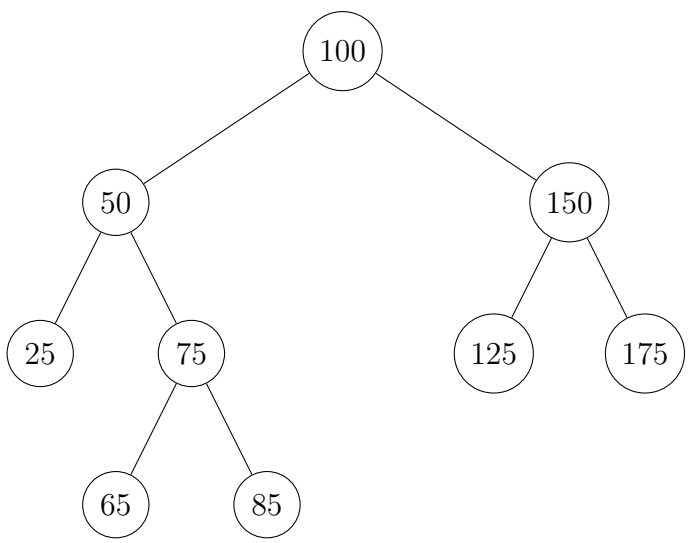



Figure 10.11: AVL Tree Example

Here, the balance factor associated with each node lies between $-1$ and $+1$. Therefore, the given tree is an AVL tree.

**Complexity of AVL Tree**

| **Operation** | **Average Case** | **Worst Case** |
|---|---|---|
| Space | $O(n)$ | $O(n)$ |
| Search | $O(\log n)$ | $O(\log n)$ |
| Insert | $O(\log n)$ | $O(\log n)$ |
| Delete | $O(\log n)$ | $O(\log n)$ |

**Why AVL Tree?**

An AVL tree controls the height of a Binary Search Tree by preventing it from becoming skewed.

In a normal BST:

- Average case time complexity is $O(\log n)$
- Worst case (skewed tree) becomes $O(n)$

In an AVL tree:

- Height is always bounded by $O(\log n)$
- All operations (search, insert, delete) take $O(\log n)$ time

### Operations on AVL Tree

| S.No. | Operation | Description |
|---|---|---|
| 1. | Insertion | Insertion in an AVL tree is performed in the same way as in a Binary Search Tree. However, after insertion, the balance factor of nodes may violate AVL properties. In such cases, the tree is rebalanced using appropriate rotations. |
| 2. | Deletion | Deletion is also performed similarly to a Binary Search Tree. However, deletion may disturb the balance of the tree. Therefore, various types of rotations are applied to rebalance the tree. |

### AVL Rotations

Rotations in an AVL tree are performed when the balance factor of any node becomes less than $-1$ or greater than $+1$.

- **LL Rotation (Left-Left)**
  Occurs when the inserted node is in the left subtree of the left child. The tree becomes left-heavy and is corrected using a **right rotation**.

- **RR Rotation (Right-Right)**
  Occurs when the inserted node is in the right subtree of the right child. The tree becomes right-heavy and is corrected using a **left rotation**.

- **LR Rotation (Left-Right)**
  Occurs when the inserted node is in the right subtree of the left child. It is corrected using two rotations: first a **left rotation**, then a **right rotation**.

- **RL Rotation (Right-Left)**
  Occurs when the inserted node is in the left subtree of the right child. It is corrected using two rotations: first a **right rotation**, then a **left rotation**.

### 10.9.5 B-Tree

A **B-Tree** is a self-balancing **m-way search tree** that is widely used in databases and file systems.

A B-Tree of order $m$ can have at most $(m-1)$ keys and $m$ children.

**Properties:**

1. Every node in a B-Tree can have at most $m$ children.
2. Every internal node (except the root) must have at least $\lceil m/2 \rceil$ children.
3. The root node must have at least two children (if it is not a leaf).
4. All leaf nodes must be at the same level.

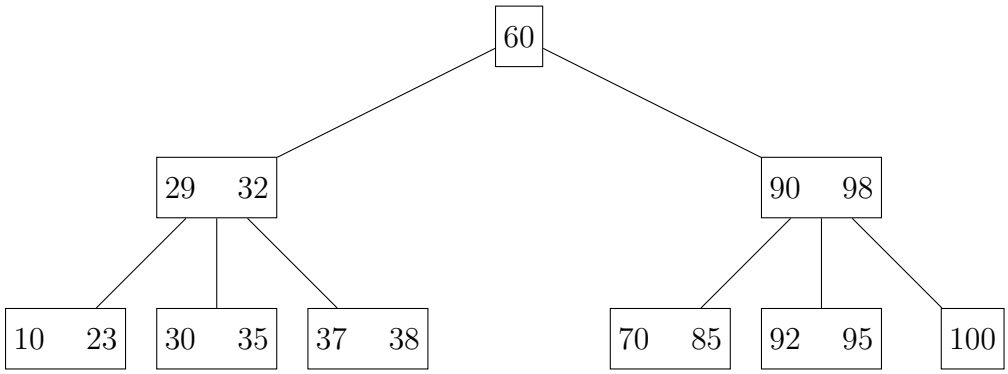



Figure 10.12: B-Tree Example

**Operations on B-Tree**

**1. Searching**

Searching in a B-Tree is similar to searching in a Binary Search Tree, but here each node contains multiple keys.

**Steps:**

1. Start from the root node.
2. Compare the search key with all keys in the current node.
3. 
   - If a match is found, return the element.
   - If the key is smaller, move to the appropriate left child.
   - If the key is greater, move to the appropriate right child.
4. Repeat the process until the element is found or the search fails.

**Time Complexity:** $O(\log n)$

**2. Insertion**

Insertion in a B-Tree is performed at the leaf node level.

**Steps:**

1. Traverse the tree to find the appropriate leaf node for insertion.
2. If the leaf node contains less than $(m-1)$ keys:
   - Insert the new element in sorted order.
3. If the leaf node already contains $(m-1)$ keys:
   - Insert the new element in sorted order.
   - Split the node into two nodes at the median.
   - Move the median element to the parent node.
4. If the parent node also becomes full:
   - Repeat the splitting process recursively.
5. If the root node splits:
   - Create a new root node.

**Applications of B-Tree**

- B-Tree is used to index data and provides fast access to actual data stored on disks.
- In large databases, data is stored on disk, and accessing it directly is time-consuming. B-Trees reduce the number of disk accesses by keeping the tree height small.
- Searching in an unindexed and unsorted database containing $n$ key values requires $O(n)$ time.
- Using B-Tree indexing, searching can be performed in $O(\log n)$ time.

### 10.9.6 B+ Tree

A **B+ Tree** is an extension of a B-Tree that supports efficient insertion, deletion, and searching operations.

In a B+ Tree:

- All actual data records are stored only in the **leaf nodes**.
- Internal nodes store only **keys** for indexing.

- All leaf nodes are linked together in the form of a **linked list**, which makes sequential access efficient.

**Advantages of B+ Tree:**

1. Records can be accessed in a predictable and equal number of disk accesses.
2. The height of the tree remains balanced and is generally smaller compared to B-Tree.
3. Supports efficient sequential as well as direct access to data.
4. Keys are used for indexing, while actual data is stored in leaf nodes.

**Difference between B-Tree and B+ Tree:**

- In B-Tree, data is stored in all nodes.
- In B+ Tree, data is stored only in leaf nodes.
- B+ Tree supports faster sequential access.

# Chapter 11

# Binary Search Trees

# Chapter 12

# Heaps

# Chapter 13

# Hashing

## 13.1 Introduction to Hashing

Hashing is a technique used to store and retrieve data efficiently using a **hash function**.

It maps a key to an index in a hash table.

**Advantages:**

- Fast insertion, deletion and searching
- Average time complexity: $O(1)$

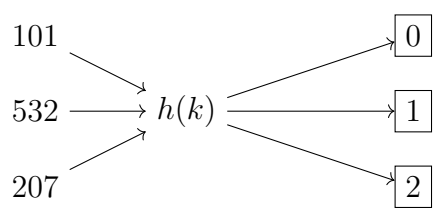



Figure 13.1: Basic Hashing Concept

—

## 13.2 Hash Function

A hash function maps a key to an index.

$$h(key) = key \bmod table_size$$

**Properties:**

- Uniform distribution
- Fast computation
- Minimizes collisions

—

## 13.3 Collision

A collision occurs when two keys map to the same index.

**Reasons:**

- Limited table size
- Poor hash function

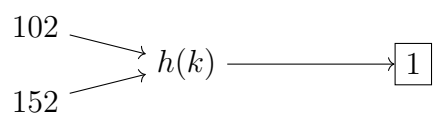



Figure 13.2: Collision Example

—

## 13.4 Collision Resolution Techniques

- Separate Chaining
- Open Addressing

—

### 13.4.1 Separate Chaining

Each index stores a linked list of elements.

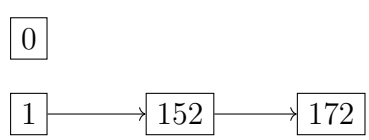



Figure 13.3: Separate Chaining

**Advantages:**

- Simple
- No overflow

—

### 13.4.2 Open Addressing

All elements are stored inside the table.

—

#### Linear Probing

$$index = (h(key) + i) \bmod table_size$$

Collision → next slot

Figure 13.4: Linear Probing

#### Disadvantage: Primary Clustering

—

#### Quadratic Probing

$$index = (h(key) + i^2) \bmod table_size$$

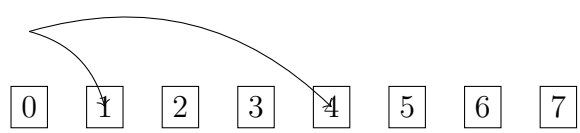



Figure 13.5: Quadratic Probing

—

#### Double Hashing

$$index = (h_1(key) + i \cdot h_2(key)) \bmod table_size$$

#### Best collision resolution method

—

## 13.5 Rehashing

Rehashing increases table size and re-inserts elements.

$$\lambda = \frac{n}{m}$$

where $n$ = elements, $m$ = table size.

—

## 13.6 Extendible Hashing

Uses directory and buckets.

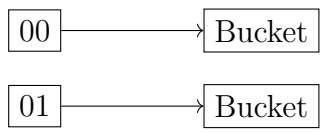



Figure 13.6: Extendible Hashing

—

## 13.7 Time Complexity

| Operation | Average Case |
|---|---|
| Insert | $O(1)$ |
| Search | $O(1)$ |
| Delete | $O(1)$ |

# Chapter 14

# Graphs

## 14.1 Introduction

A **graph** is a data structure consisting of a set of vertices and edges that connect these vertices.

**Definition:** A graph $G$ can be defined as an ordered pair:

$$G = (V, E)$$

where:

- $V(G)$ represents the set of vertices
- $E(G)$ represents the set of edges connecting the vertices

### 14.1.1 Directed and Undirected Graph

A graph can be either **directed** or **undirected**.

In an **undirected graph**, edges do not have any direction. In a **directed graph**, edges have a specific direction.

**Undirected Graph**

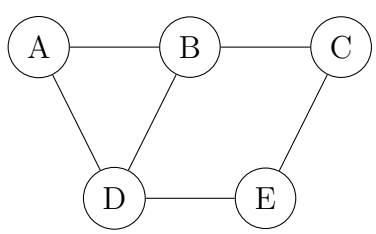



Figure 14.1: Undirected Graph

In an undirected graph, edges are not associated with any direction.

**Directed Graph**

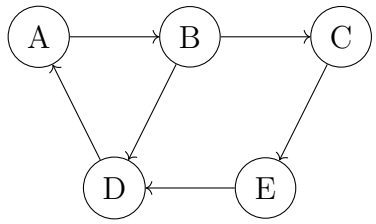



Figure 14.2: Directed Graph

In a directed graph, edges represent ordered pairs and have a specific direction.

## 14.2 Graph Terminology

1. **Path**
   A path is a sequence of vertices that are connected by edges, followed in order from an initial vertex $v_0$ to a terminal vertex $v_n$.

2. **Closed Path**
   A path is called a closed path if the initial vertex and terminal vertex are the same, i.e., $v_0 = v_n$.

3. **Simple Path**
   A path in which all vertices are distinct (except possibly the first and last in case of a closed path) is called a simple path.

4. **Cycle**
   A cycle is a path in which no vertices (and no edges) are repeated, except the first and last vertex.

5. **Connected Graph**
   A graph is said to be connected if there exists a path between every pair of vertices. Such a graph has no isolated nodes.

6. **Complete Graph**
   A graph in which every vertex is connected to every other vertex is called a complete graph. A complete graph with $n$ vertices has:

   $$\frac{n(n-1)}{2}$$

   edges.

7. **Weighted Graph**
   A graph in which each edge is assigned a weight (such as cost, distance, or time) is called a weighted graph.

8. **Digraph**
   A digraph (directed graph) is a graph in which each edge has a direction, and traversal is possible only in that direction.

9. **Loop**
   An edge that connects a vertex to itself is called a loop.

10. **Adjacent Nodes**
    Two vertices $u$ and $v$ are said to be adjacent if there is an edge connecting them. They are also called neighbors.

11. **Degree of a Node**
    The degree of a node is the number of edges connected to it. A node with degree 0 is called an isolated node.

## 14.3 Graph Representation

Graph representation refers to techniques used to store a graph in computer memory.

### 14.3.1 Adjacency Matrix Representation(Sequential)

In this representation, a graph with $n$ vertices is stored using an $n \times n$ matrix.

An entry $M[i][j] = 1$ if there is an edge between vertex $v_i$ and $v_j$, otherwise 0.

**Undirected Graph**

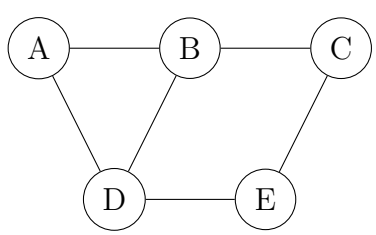



Figure 14.3: Undirected Graph

| | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 1 | 0 |
| B | 1 | 0 | 1 | 1 | 0 |
| C | 0 | 1 | 0 | 0 | 1 |
| D | 1 | 1 | 0 | 0 | 1 |
| E | 0 | 0 | 1 | 1 | 0 |

**Directed Graph**

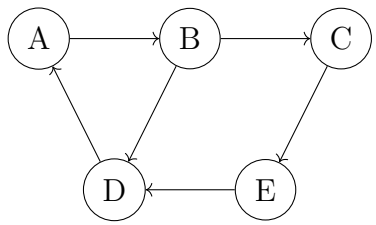



Figure 14.4: Directed Graph

| | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 0 |
| B | 0 | 0 | 1 | 1 | 0 |
| C | 0 | 0 | 0 | 0 | 1 |
| D | 1 | 0 | 0 | 0 | 0 |
| E | 0 | 0 | 0 | 1 | 0 |

**Weighted Directed Graph**

In a weighted graph, edges carry weights instead of just 1.

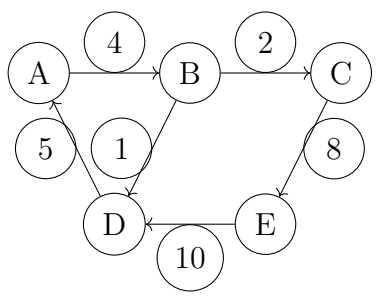



Figure 14.5: Weighted Directed Graph

| | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 4 | 0 | 0 | 0 |
| B | 0 | 0 | 2 | 1 | 0 |
| C | 0 | 0 | 0 | 0 | 8 |
| D | 5 | 0 | 0 | 0 | 0 |
| E | 0 | 0 | 0 | 10 | 0 |

### 14.3.2 Adjacency List (Linked Representation)

In this representation, each vertex maintains a list of adjacent vertices. Each node stores the vertex value and a pointer to the next adjacent node.

**Undirected Graph**

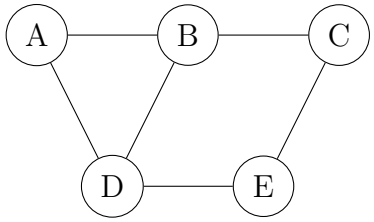



Figure 14.6: Undirected Graph

A → B → D → NULL
B → A → C → D → NULL
C → B → E → NULL
D → A → B → E → NULL
E → D → C → NULL

**Directed Graph**

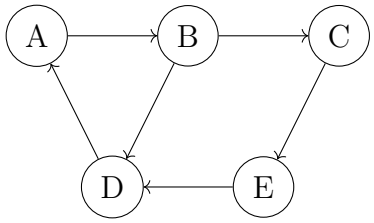



Figure 14.7: Directed Graph

A → B → NULL
B → C → D → NULL
C → E → NULL
D → A → NULL
E → D → NULL

**Note:** In a directed graph, the sum of the lengths of all adjacency lists is equal to the number of edges in the graph.

## 14.4 Graph Traversal Algorithms

Graph traversal refers to the process of visiting all the vertices of a graph.

There are two standard methods for graph traversal:

- Breadth First Search (BFS)
- Depth First Search (DFS)

### 14.4.1 Breadth First Search (BFS)

Breadth First Search is a graph traversal algorithm that starts from a root node and explores all its neighboring nodes first, before moving to the next level of nodes.

It follows a **level-order traversal** approach and uses a **queue** data structure.

#### Algorithm

1. Mark all nodes as unvisited.
2. Create an empty queue $Q$.
3. Choose a starting node and:
   - Mark it as visited
   - Enqueue it into $Q$
4. While the queue is not empty:
   - Dequeue a node $N$
   - Process (visit) $N$
   - Enqueue all unvisited neighbors of $N$ and mark them as visited
5. Repeat until all nodes are visited.

#### Example

Adjacency List:

- A: B, D
- B: C, F
- C: E, G
- D: F
- E: B, F
- F: A
- G: E

Starting from node A, BFS traversal is:

$$A \to B \to D \to C \to F \to E \to G$$

### 14.4.2 Depth First Search (DFS)

Depth First Search (DFS) is a graph traversal algorithm that starts from an initial node and explores as far as possible along each branch before backtracking.

DFS uses a **stack** data structure (or recursion).

**Algorithm (Using Stack)**

1. Mark all nodes as unvisited.
2. Create an empty stack $S$.
3. Push the starting node onto the stack and mark it as visited.
4. While the stack is not empty:
   - Pop a node $N$ from the stack
   - Process (visit) $N$
   - Push all unvisited neighbors of $N$ onto the stack and mark them as visited
5. Repeat until all nodes are visited.

**Algorithm (Recursive)**

1. Mark the current node as visited.
2. Process the node.
3. Recursively visit all unvisited neighbors.

## 14.5 Spanning Tree

If a graph $G(V, E)$ has $V$ vertices and $E$ edges, then a **spanning tree** of the graph is a subgraph that:

- Contains all the vertices of the graph
- Has exactly $(V - 1)$ edges
- Does not contain any cycles

### 14.5.1 Properties of Spanning Tree

- A connected graph can have more than one spanning tree.

- All spanning trees of a graph have the same number of vertices and exactly $(V - 1)$ edges.
- A spanning tree does not contain any cycles.
- If any edge is removed from a spanning tree, it becomes disconnected.
- If an edge is added to a spanning tree, it creates a cycle.

## 14.6 Minimum Spanning Tree (MST)

A **Minimum Spanning Tree** is a spanning tree in which the sum of all edge weights is minimum.

### 14.6.1 Properties of Minimum Spanning Tree

- If all edge weights are distinct, then the MST is unique.
- If multiple edges have the same weight, there may be more than one MST.
- MST always contains $(V - 1)$ edges.
- MST is a subset of edges that connects all vertices with minimum total cost.

### 14.6.2 Kruskal's Algorithm

Kruskal's algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, weighted graph.

It works by selecting edges in increasing order of their weights and adding them to the MST, ensuring that no cycle is formed.

#### Algorithm

1. Sort all edges in non-decreasing order of their weights.
2. Initialize the MST as an empty set.
3. For each edge $(u, v)$ in sorted order:
   - If adding the edge does not form a cycle:
     - Add the edge to the MST
4. Repeat until MST contains $(V - 1)$ edges.

**Time Complexity:** $O(E \log E)$

### 14.6.3 Prim's Algorithm

Prim's algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, weighted graph.

It starts from any arbitrary vertex and grows the MST by adding the smallest edge that connects a vertex in the tree to a vertex outside the tree.

**Algorithm**

1. Start with any vertex as the initial node.
2. Mark the starting node as visited.
3. Select the minimum weight edge that connects a visited vertex to an unvisited vertex.
4. Add the selected edge and vertex to the MST.
5. Repeat steps 3 and 4 until all vertices are included in the MST.

**Time Complexity:** $O(E \log V)$ (using priority queue)